



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Environment
Actuators
Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

The Environment (E) state is represented by `self.width`, `self.height`, `self.agent_pos`, `self.walls`, `self.food_positions`, and `self.opponents`. These variables define the grid, agent location, obstacles, food, and other agents in the game world.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

besides the main agent, these opponents move independently and can affect the game by interacting with the agent, such as causing collisions and changing the score. Therefore the environment contains multiple active agents.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

the environment over time. It includes the agent's past percepts and helps the agent choose its next action.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

framework. It provides the agent with information about the current environment, such as the agent's position, opponent positions, food detection, wall detection, collision status, score, and remaining food, which the agent uses to decide its next action.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

The environment is Partially Observable because `get_percept()` only provides some information about the environment. The agent can see its position, opponents, score, and nearby conditions, but it cannot see the full grid state, such as all food and wall

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The PEAS component being updated is the Performance Measure. When `execute_action()` deducts points for hitting a wall, it changes the agent's score, which represents how well agent is performing in its environment.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is important because the metric should evaluate the agent's actual interaction with the environment, not its internal effort. Hitting a wall shows the effect of the agent's action on the external world.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

- Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

environment Partially Observable. The agent cannot access the complete state of the environment because it cannot detect the trap locations.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding `smells_toxin` gives the agent extra information about possible dangers in the environment. This helps the agent make safer decisions, avoid toxic areas, and choose actions that maximize its expected utility by reducing penalties and improving survival.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World exploit showed that a faulty metric can make an agent repeat useless actions (like cleaning already-clean squares) just to gain rewards instead of achieving the real goal.

Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING